

Übung 7: Dokumentation, R-Glossar und Shiny-Backup-Maske

Zur Selbstkontrolle: Für 2024 liefert die Funktion zehn Bereiche mit jeweils vier Quartalen. Die Jahressumme über alle Bereiche beträgt 3.827.201,0 Mio. EUR.

Voraussetzungen

- Arbeitsverzeichnis ist das RStudio-Projekt `projekt/`.
- Datenzugriff relativ als `data/...`
- Benötigte Pakete (alle installiert): `dplyr`, `readr`, `ggplot2`, `shiny`.

Dateien

Datei	Zweck
<code>R/lab7_funktionen.R</code>	Ihre Startdatei mit # TODO-Lücken
<code>app.R</code>	Shiny-Backup-Maske
R-Glossar im Ordner <code>R</code>	Gemeinsame Wissenssammlung

Schritt 1: Funktion und Dokumentation

Öffnen Sie `R/lab7_funktionen.R`. Sie sehen das Gerüst der Funktion `bws_zusammenfassung(daten, jahr)`. Sie soll je Wirtschaftsbereich die vier Quartalswerte eines Jahres zur Jahressumme aufaddieren.

Beachten Sie den roxygen-Kopf darüber. Jede Zeile beginnt mit `#'`:

```
#' TODO: kurzer Titel
#
#' @param daten TODO: Struktur und benötigte Spalten beschreiben.
#' @param jahr TODO: erwarteten Wert beschreiben.
#' @return TODO: Rückgabe beschreiben.
#' @examples
#' bws_zusammenfassung(daten, 2024)
```

- `#'` (erste Zeile): Titel der Funktion, kurz und konkret.
- `@param`: ein Eintrag je Argument, was es ist und welcher Typ erwartet wird.
- `@return`: was zurückkommt (Struktur, Spalten, Einheit).
- `@examples`: lauffähige Aufrufbeispiele.

Mit `roxygen2` kann daraus später eine Hilfeseite erzeugt werden.

Schritt 2: Eingaben mit `stopifnot()` absichern (TODO 1)

`stopifnot()` prüft mehrere Bedingungen und bricht beim ersten `FALSE` ab.

Vervollständigen Sie **TODO 1**. Prüfen Sie mindestens:

```
stopifnot(
  # TODO: daten ist ein data.frame,
  # TODO: benötigte Spalten sind vorhanden,
  # TODO: wert ist numerisch,
  # TODO: jahr hat Länge 1 und kommt in den Daten vor
)
```

Hinweis. `all(... %in% names(daten))` prüft, ob alle benötigten Spalten da sind. `length(jahr) == 1` verhindert versehentliche Vektoren. Die letzte Zeile stellt sicher, dass das gewünschte Jahr überhaupt in den Daten vorkommt.

Schritt 3: Aggregieren (TODO 2)

Vervollständigen Sie **TODO 2**: filtern, gruppieren, summieren, sortieren.

```
bws_zusammenfassung <- function(daten, jahr) {  
  # TODO: Jahr filtern, nach Bereich gruppieren, Jahressumme und  
  #       Quartalsanzahl berechnen und das Ergebnis sortieren.  
  NULL  
}
```

Hinweis. `.data$jahr == .env$jahr` unterscheidet die Spalte `jahr` (`.data`) vom Funktionsargument `jahr` (`.env`). So vermeiden Sie die klassische Namenskollision, bei der dplyr nicht weiß, welches `jahr` gemeint ist. `n_quartale` zählt mit, wie viele Quartale eingeflossen sind: bei einer vollständigen Lieferung sollten das vier sein.

Schritt 4: Funktion testen

Testen Sie Ihre Datei aus dem Projektordner:

```
Rscript R/lab7_funktionen.R
```

Der Aufruf definiert die Funktion und wendet sie auf `data/lieferung_2024q4.csv` an. Sie sehen eine Tabelle mit einer Zeile je Bereich, absteigend nach `jahressumme`. Für 2024 steht BE mit einer Jahressumme von 896.778 Mio. EUR an erster Stelle.

Schritt 5: Das R-Glossar pflegen

Öffnen Sie das R-Glossar im Ordner `R`. Es enthält Startbeispiele zu wiederkehrenden VGR-Problemen (Dezimalkomma-Import, Klassifikationswechsel, Override-Tabelle, Join-Schlüsseltreue), jeweils nach demselben Muster: **Problem, Symptom, Lösung, Codeschnipsel**.

Aufgabe. Ergänzen Sie einen eigenen Abschnitt zu einem Problem, das Sie in diesem Kurs getroffen haben (zum Beispiel Excel-Import mit `skip`, PDF-Parsing mit `pdftools` oder Abweichungsanalyse mit `waldo`). Halten Sie sich an die vier Punkte und fügen Sie einen kurzen, lauffähigen Codeschnipsel an.

Schritt 6: Die Shiny-Backup-Maske starten

`app.R` enthält die Shiny-Maske. Starten Sie sie aus dem Projektordner:

```
# in der R-Konsole, Arbeitsverzeichnis = projekt/  
shiny::runApp("app.R")
```

In RStudio können Sie alternativ `app.R` öffnen und oben rechts auf **Run App** klicken. Die App öffnet sich im Browser oder im RStudio-Viewer. Beenden mit `Strg+C` in der Konsole oder dem Stopp-Symbol.

Funktionen:

- `selectInput` **Jahr** und `selectInput` **Wirtschaftsbereich** zur Auswahl,
- `actionButton` **Anzeigen**: rechnet erst auf Klick (reaktiv),
- `tableOutput`: die vier Quartalswerte der Auswahl,
- `plotOutput`: ggplot-Zeitreihe über die Quartale,
- `downloadButton`: das gefilterte Ergebnis als CSV.

Werfen Sie in `app.R` einen Blick auf die `server`-Funktion: `eventReactive(input$anzeigen, ...)` hält den Auszug reaktiv; `renderTable`, `renderPlot` und `downloadHandler` verwenden ihn.

Stolperfalle: Die App und die Funktion erwarten das Arbeitsverzeichnis `projekt/`. Starten Sie aus einem anderen Ordner, finden die relativen Pfade unter `data/...` ihre Dateien nicht.

Weiterführend (optional)

- Erweitern Sie `app.R`, sodass die Jahressumme aus `bws_zusammenfassung()` als zusätzliche Kennzahl angezeigt wird.
- Geben Sie der App einen zweiten `selectInput`, um mehrere Bereiche zu vergleichen (Tipp: `selectInput(..., multiple = TRUE)` und `geom_line` je Bereich mit `color = bereich_name`).